

WIDO

Moteur Sémantique pour JeuxDeMots

Langage de Requêtes Formelles avec Heuristiques d'Optimisation

Rapport de projet TER
Master Informatique — Spécialité ICO
Université de Montpellier, 2025–2026

Mohamed TEDJINI
Maurice DJOBO

Encadrant : à préciser

Mai 2026

Abstract

WIDO est un moteur de recherche sémantique permettant d'interroger le graphe de connaissances JEUXDEMOTS (JDM) via un langage de requêtes formel à variables. L'utilisateur formule des requêtes telles que (**\$x** **r_isa** **animal**) **ET** (**\$x** **r_has_part** **queue**), que le moteur parse en arbre de décision, optimise par des heuristiques structurelles et de cardinalité, puis exécute en interrogeant l'API JDM v0. Les résultats sont croisés par jointures ensemblistes basées sur les identifiants des nœuds, classés par score de pertinence, et présentés avec leurs preuves dans une interface web glassmorphe.

Contents

1	Introduction	3
1.1	Problématique	3
1.2	Objectifs	3
1.3	Plan du rapport	3
2	Contexte et données	4
2.1	JeuxDeMots	4
2.2	Structure du graphe JDM	4
2.3	API JDM v0	4
3	État de l’art	6
3.1	Moteurs de recherche sémantique	6
3.2	Spécificité de JDM	6
3.3	Optimisation de requêtes	6
3.4	Comparaison par identifiant	6
4	Architecture du moteur WIDO	7
4.1	Vue d’ensemble	7
4.2	Technologies utilisées	8
4.3	Structure des fichiers	8
5	Parseur syntaxique	9
5.1	Langage de requêtes WIDO	9
5.2	Exemples de requêtes	9
5.3	Structure de l’AST	9
5.4	Gestion des erreurs syntaxiques	10
6	Heuristiques d’optimisation	11
6.1	Principe général	11
6.2	Heuristique structurelle	11
6.3	Heuristique de cardinalité	11
6.4	Mise à jour dynamique des variables ancrées	12
6.5	Plan d’exécution (exemple)	12
7	Moteur d’exécution	13
7.1	Principe d’exécution	13
7.2	Trois cas de traitement d’une clause de relation	13
7.2.1	Cas 1 : Ancrage initial (Constante → Variable)	13
7.2.2	Cas 2 : Vérification (Variable ancrée → Constante)	13

7.2.3	Cas 3 : Jointure (Variable $\rightarrow$ Variable)	13
7.3	Comparaison par ID JDM	13
7.4	Scores et preuves	14
7.5	Traitement des opérateurs OU	14
8	Client API et cache	15
8.1	Pagination de l'API JDM	15
8.2	Cache disque MD5	15
8.3	Gestion des erreurs API	16
9	Interface utilisateur	17
9.1	Design glassmorphe	17
9.2	Zone de saisie et résultats	17
9.3	Panneau de diagnostic	17
9.4	Gestion des erreurs	17
10	Évaluation et benchmark	18
10.1	Protocole	18
10.2	Requêtes testées	18
10.3	Analyse des résultats	18
11	Limites et perspectives	20
11.1	Limites actuelles	20
11.2	Perspectives d'amélioration	20
12	Conclusion	21

Chapter 1

Introduction

Ce projet de Travail d'Étude et de Recherche (TER) s'inscrit dans le cadre du Master Informatique, spécialité ICO. Il porte sur la conception et l'implémentation d'un moteur de recherche sémantique nommé WIDO, capable d'interroger le graphe de connaissances JEUXDEMOTS via un langage de requêtes formel à variables.

1.1 Problématique

Les graphes de connaissances comme JEUXDEMOTS contiennent des millions de relations entre concepts. L'enjeu est de permettre à un utilisateur non expert de formuler des requêtes complexes en combinant plusieurs relations, opérateurs logiques et contraintes textuelles, sans avoir à connaître la structure interne de la base.

1.2 Objectifs

Les objectifs de ce projet sont les suivants :

1. Définir un langage de requêtes formel adapté à JDM ;
2. Implémenter un parseur syntaxique produisant un arbre de décision (AST) ;
3. Concevoir un planificateur d'exécution basé sur des heuristiques structurelles et de cardinalité ;
4. Réaliser un moteur d'exécution qui interroge l'API JDM v0 et croiser les résultats par jointures sur les identifiants des nœuds ;
5. Développer une interface web permettant la saisie de requêtes et la visualisation des résultats avec leurs preuves ;
6. Évaluer les performances du moteur via un benchmark automatique.

1.3 Plan du rapport

Le chapitre 2 présente le contexte du projet et le graphe JeuxDeMots. Le chapitre 4 décrit l'architecture globale du moteur. Les chapitres 5 à 8 détaillent chaque composant. Le chapitre 10 présente les résultats du benchmark. Enfin, les chapitres 11 et 12 discutent des limites et ouvrent des pistes d'amélioration.

Chapter 2

Contexte et données

2.1 JeuxDeMots

JEUXDEMOTS est un jeu en ligne sérieux développé par le LIRMM (Montpellier) dont le but est de construire collaborativement un réseau lexical du français. Les joueurs associent des mots à des propriétés, des catégories, des synonymes, des actions possibles, etc. Le résultat est un graphe de connaissances contenant des millions de nœuds et de relations.

2.2 Structure du graphe JDM

Chaque **nœud** représente un concept (mot, expression, syntagme) et est identifié par un entier unique (ID JDM). Chaque **relation** est un triplet (nœud source, type de relation, nœud cible) associé à un poids (score de confiance) allant de 1 à plusieurs milliers.

Les types de relations principaux utilisés dans WIDO sont :

Nom	ID	Signification
<code>r_isa</code>	6	Appartient à la catégorie
<code>r_has_part</code>	9	Possède la partie
<code>r_can_eat</code>	43	Peut manger
<code>r_color</code>	42	A pour couleur
<code>r_carac</code>	12	A pour caractéristique
<code>r_synt_synon</code>	1	Synonyme syntaxique

Table 2.1: Types de relations JDM utilisés dans WIDO

2.3 API JDM v0

L'API REST publique du LIRMM expose les données JDM via des endpoints structurés. Elle supporte la pagination native (`limit + offset`) et le filtrage par type de relation (`types_ids`) et poids minimum (`min_weight`). Le tableau [2.2](#) résume les endpoints exploités dans WIDO.

Endpoint	Usage
<code>/v0/node_by_name/{name}</code>	Résoudre un nom en nœud JDM
<code>/v0/relations/from/{name}</code>	Récupérer les relations sortantes
<code>/v0/relations/to/{name}</code>	Récupérer les relations entrantes
<code>/v0/relations/from_by_id/{id}</code>	Relations sortantes par ID
<code>/v0/relations/to_by_id/{id}</code>	Relations entrantes par ID
<code>/v0/relations/from_by_id/{id1}/to_by_id/{id2}</code>	Vérification d'un couple

Table 2.2: Endpoints API JDM utilisés par WIDO

Chapter 3

État de l’art

3.1 Moteurs de recherche sémantique

Les moteurs de recherche sémantique sur des graphes de connaissances ont été largement étudiés dans le contexte du Web sémantique. SPARQL [1] est le standard pour interroger des graphes RDF (Resource Description Framework). Des moteurs comme Virtuoso ou GraphDB permettent d’exécuter des requêtes SPARQL sur des bases de connaissances volumineuses.

3.2 Spécificité de JDM

À la différence de DBpedia ou Wikidata, JDM est un graphe informel construit collaborativement. Il n’expose pas nativement de point SPARQL, d’où la nécessité d’une couche d’abstraction comme WIDO.

3.3 Optimisation de requêtes

L’optimisation de l’ordre d’exécution des jointures est un problème classique en bases de données relationnelles (algorithme de Selinger [3]). Dans le contexte des graphes, les heuristiques de cardinalité permettent d’estimer le coût d’une jointure avant de l’exécuter. Notre approche combine une heuristique structurelle (coût syntaxique) et une estimation de cardinalité par sondage de l’API.

3.4 Comparaison par identifiant

Dans un graphe de connaissances, comparer deux nœuds par leur identifiant plutôt que par leur nom est une pratique recommandée pour éviter les ambiguïtés dues aux homonymes et aux variantes typographiques [2].

Chapter 4

Architecture du moteur WIDO

4.1 Vue d'ensemble

Le moteur WIDO suit un pipeline en cinq étapes (figure 4.1) :

1. **Parsing** : la requête textuelle est transformée en arbre de décision (AST) par le parseur ;
2. **Estimation de cardinalité** : pour chaque clause, un appel API sonde estime le nombre de résultats ;
3. **Planification** : le planificateur trie les clauses ET dans l'ordre d'exécution optimal ;
4. **Exécution** : le moteur interroge l'API JDM et croise les résultats par jointures sur les IDs ;
5. **Présentation** : les résultats sont triés par score, enrichis de preuves, et affichés.

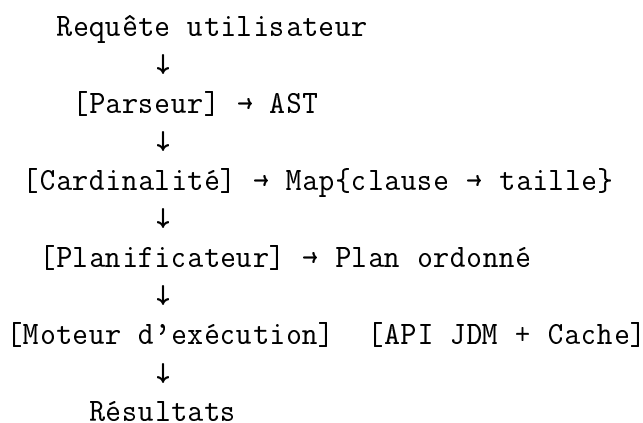


Figure 4.1: Pipeline de traitement d'une requête WIDO

4.2 Technologies utilisées

- **Environnement d'exécution** : Node.js v14+
- **Serveur HTTP** : Express.js
- **Client HTTP** : Axios (requêtes vers l'API JDM)
- **Cache disque** : fs-extra, fichiers JSON indexés par hash MD5
- **Interface** : HTML5 + CSS3 (glassmorphe) + JavaScript vanilla

4.3 Structure des fichiers

Fichier	Rôle
<code>node.js</code>	Point d'entrée, routage Express, orchestration
<code>parseur.js</code>	Analyse syntaxique → AST
<code>heuristiques.js</code>	Planification par complexité et cardinalité
<code>moteurExecution.js</code>	Exécution, jointures, intersections
<code>jdmApi.js</code>	Client API JDM + pagination + cache
<code>cacheManager.js</code>	Cache disque MD5 persistant
<code>benchmark.js</code>	Tests automatisés + métriques
<code>verify_logic.js</code>	Tests de validation du moteur

Table 4.1: Fichiers du projet WIDO

Chapter 5

Parseur syntaxique

5.1 Langage de requêtes WIDO

Le langage WIDO permet d'exprimer des requêtes formelles à variables sur le graphe JDM. Sa grammaire (simplifiée) est la suivante :

```
requête ::= expr
expr     ::= terme (ET terme)*
terme    ::= facteur (OU facteur)*
facteur  ::= '(' clause ')'
          | '(' expr ')'
clause   ::= '(' sujet relation objet ')'
          | '(' variable '=' filtre ')'
sujet    ::= variable | constante
objet    ::= variable | constante
variable ::= '$' identifiant
filtre   ::= [a-z%]+
```

5.2 Exemples de requêtes

```
1 ($x r_isa animal)
2 ($x r_isa animal) ET ($x = ch%)
3 ($x r_isa artiste) ET (($x = ba%) OU ($x = Ba%))
4 ($x r_isa animal) ET ($y r_isa animal) ET ($x r_can_eat $y)
5 (chat r_has_part $y) ET ($y r_isa $z)
```

Listing 5.1: Exemples de requêtes WIDO

5.3 Structure de l'AST

Le parseur produit un arbre de décision récursif. Chaque nœud interne représente un opérateur logique (ET/OU), chaque feuille est une clause atomique (relation ou filtre).

```
1 {
2   "type": "NOEUD_LOGIQUE",
```

```
3  "opérateur": "ET",
4  "gauche": {
5    "type": "CLAUSE_RELATION",
6    "variable": "$x",
7    "relation": "r_isa",
8    "cible": "animal"
9  },
10 "droite": {
11   "type": "CLAUSE_FILTRE",
12   "variable": "$x",
13   "filtre": "ch%"
14 }
15 }
```

Listing 5.2: AST produit pour ($\$x$ r_isa animal) ET ($\$x$ = ch%)

5.4 Gestion des erreurs syntaxiques

Lorsqu'une clause est incomplète (ex: ($\$x$ r_isa)), le parseur lève une exception qui est capturée par le serveur. Celui-ci retourne une réponse JSON propre avec **statut**: "Erreur" et un message explicatif, sans jamais retourner de HTTP 500 ni de champs **undefined**.

Chapter 6

Heuristiques d'optimisation

6.1 Principe général

Pour les requêtes conjonctives (ET), l'ordre d'exécution des clauses influe directement sur le nombre d'appels API et la taille des ensembles intermédiaires. L'objectif du planificateur est de minimiser cet espace de recherche en exécutant d'abord les clauses les plus *sélectives* (qui retournent peu de résultats).

6.2 Heuristique structurelle

La première heuristique est purement syntaxique : elle attribue un score de complexité à chaque clause selon sa structure et les variables déjà ancrées à ce stade de l'exécution.

Score	Type de clause	Exemple
0.5	Filtre sur variable ancrée	<code>\$x = ch%</code> (après ancrage de <code>\$x</code>)
1	Ancrage par constante	<code>(\$x r_isa animal)</code>
2	Vérification (2 vars ancrées)	Jointure finale
5	Exploration ciblée (1 var ancrée)	<code>(\$x r_can_eat \$y)</code> si <code>\$x</code> connu
8	Filtre avant ancrage	<code>\$x = ba%</code> (avant ancrage)
10	Jointure sans ancrage	Interdit en première position

Table 6.1: Scores de complexité structurelle des clauses WIDO

6.3 Heuristique de cardinalité

La deuxième heuristique estime la cardinalité (taille de la liste de résultats) de chaque clause d'ancrage par constante en effectuant un appel API sonde avec `limit=500`. Si la liste est de taille $n < 500$, la cardinalité est n . Sinon, elle est notée ≥ 500 (grande liste).

Exemple d'application : Pour la requête `($x r_isa animal) ET ($x r_has_part queue)`,
si `r_isa animal` renvoie ≥ 500 résultats et `r_has_part queue` en renvoie 105,

le planificateur exécute d'abord `r_has_part queue` (plus sélectif), puis filtre les résultats.

6.4 Mise à jour dynamique des variables ancrées

Après l'exécution de chaque clause, le planificateur met à jour l'ensemble des variables déjà résolues. Cela modifie le score de complexité des clauses suivantes et permet de détecter qu'une jointure `Var→Var` peut désormais s'appuyer sur une variable ancrée.

6.5 Plan d'exécution (exemple)

Pour la requête `($x r_isa animal) ET ($y r_isa animal) ET ($x r_can_eat $y)` :

Rang	Clause	Complexité	Raison
1	<code>(\$x r_isa animal)</code>	1	Ancrage initial — cardinalité ≥ 500
2	<code>(\$y r_isa animal)</code>	1	Ancrage initial — cardinalité ≥ 500
3	<code>(\$x r_can_eat \$y)</code>	2	Vérification (<code>\$x</code> et <code>\$y</code> ancrés)

Table 6.2: Plan d'exécution de la requête officielle à 2 variables

Chapter 7

Moteur d'exécution

7.1 Principe d'exécution

Le moteur d'exécution parcourt le plan produit par le planificateur et maintient un **contexte** : une liste de tuples partiels représentant les liaisons variable $\rightarrow$ nœud JDM découvertes jusqu'à présent.

7.2 Trois cas de traitement d'une clause de relation

7.2.1 Cas 1 : Ancrage initial (Constante $\rightarrow$ Variable)

La variable cible n'est pas encore ancrée. Le moteur interroge l'API JDM (`/from/` ou `/to/`) pour obtenir la liste des nœuds satisfaisant la clause, et crée de nouveaux tuples.

Exemple : `($x r_isa animal)  $\rightarrow$  appel à /v0/relations/to/animal?types_ids=6.`

Résultat : `[{$x: {id: 39642, name: "chat"}}, {$x: {id: ..., name: "chien"}}, ...]`

7.2.2 Cas 2 : Vérification (Variable ancrée $\rightarrow$ Constante)

La variable source est déjà ancrée. Le moteur récupère la liste valide depuis l'API, construit un **Set** d'IDs, et filtre le contexte en $O(1)$.

Exemple : `($x r_has_part queue)` après ancrage de `$x` $\rightarrow$ appel unique + filtrage local.

7.2.3 Cas 3 : Jointure (Variable $\rightarrow$ Variable)

Les deux variables sont des variables, au moins l'une étant déjà ancrée. Le moteur itère sur les candidats de la variable ancrée, effectue un appel API par candidat (via `from_by_id` ou `to_by_id`), et conserve les couples valides.

7.3 Comparaison par ID JDM

Toutes les intersections et jointures utilisent les **identifiants numériques** des nœuds JDM, et non leurs noms textuels. Cette stratégie garantit des comparaisons exactes en

$O(1)$ avec un `Set<id>`, et évite les ambiguïtés dues aux homonymes ou aux variantes typographiques.

```
1 // Construction d'un Set d'IDs valides (en  $O(n)$ )
2 const validIds = new Set(apiResults.map(r => r.id));
3
4 // Filtrage local en  $O(1)$  par ID
5 return contexte.filter(tuple =>
6     tuple[$x] && validIds.has(tuple[$x].id)
7 );
```

Listing 7.1: Comparaison par ID dans le moteur d'exécution

7.4 Scores et preuves

Chaque tuple porte deux métadonnées :

- `--score` : somme des poids des relations traversées ;
- `--preuves` : liste des clauses qui justifient l'inclusion de ce tuple dans le résultat.

Les résultats finaux sont triés par score décroissant.

7.5 Traitement des opérateurs OU

Pour les branches OU, le moteur exécute chaque branche indépendamment et calcule l'union des résultats. Les doublons (même combinaison d'IDs) sont éliminés en conservant le tuple de score maximal.

Chapter 8

Client API et cache

8.1 Pagination de l'API JDM

L'API JDM retourne au plus n résultats par appel. Pour récupérer des listes complètes, WIDO implémente une boucle de pagination native avec les paramètres `limit` et `offset`.

```
1 let offset = 0, page = 0;
2 while (true) {
3     if (page >= maxPages) break;           // Limite de s c urit
4     if (results.length >= maxTotal) break;
5
6     const url = `${base}?types_ids=${relId}
7                 &limit=${limit}&offset=${offset}`;
8     const data = await apiCall(url);
9
10    results.push(...data.relations);
11    page++; offset += limit;
12
13    if (data.relations.length < limit) break; // Derniere page
14 }
```

Listing 8.1: Boucle de pagination (extrait simplifié de `jdmApi.js`)

Les paramètres de sécurité sont : `limit=1000`, `maxPages=5`, `maxTotal=5000`.

8.2 Cache disque MD5

Le cache disque évite les appels redondants à l'API JDM. Chaque clé de cache est un hash MD5 de l'URL ou des paramètres de la requête. Les données sont stockées en JSON sur le disque.

Type de cache	Clé
Résolution de nœud	<code>node_by_name:{name}</code>
Relations paginées	<code>relsPaged:{dir}:{node}:{rel}...</code>
Relations par ID	<code>relsPaged:{dir}:id:{nodeId}:{rel}...</code>
Cardinalité estimée	<code>cardinality:{dir}:{node}:{rel}...</code>
Vérification de couple	<code>check:{id1}:{rel}:{id2}</code>

Table 8.1: Structure des clés de cache WIDO

8.3 Gestion des erreurs API

- **HTTP 404** : nœud ou relation introuvable → résultat vide (cas légitime)
- **HTTP 500** : erreur serveur JDM → retry 1× après 500ms
- **Timeout (>15s)** : appel abandonné, warning remonté à l'interface
- **Erreur réseau** : log et continuation avec les données disponibles

La résilience du moteur repose sur ces mécanismes : une clause qui échoue ne bloque pas les autres.

Chapter 9

Interface utilisateur

9.1 Design glassmorphe

L'interface web adopte un style glassmorphe : fond sombre, éléments translucides avec `backdrop-filter: blur()`, typographie claire.

9.2 Zone de saisie et résultats

L'utilisateur saisit sa requête. La touche Entrée déclenche la recherche. Les résultats affichent : variables liées, score de pertinence, et preuves (bouton toggle).

9.3 Panneau de diagnostic

L'interface affiche : statut, warnings, joinStats (candidats testés, couples trouvés), pagination (pages et relations récupérées), AST.

9.4 Gestion des erreurs

Quand statut: "Erreur", bannière rouge avec message. Aucun `undefined` affiché.

Chapter 10

Évaluation et benchmark

10.1 Protocole

Le benchmark automatique exécute 14 requêtes représentatives sur le serveur en cours d'exécution et mesure : durée, appels API, cache, pagination, joinStats, et résultats.

10.2 Requêtes testées

#	Requête	Description
1	<code>(\$x r_isa animal)</code>	Simple, ancrage
2	<code>(\$x r_isa animal) ET (\$x = ch%)</code>	Filtre texte
3	<code>(\$x r_isa animal) ET (\$x r_has_part queue)</code>	Jointure ET
4	<code>(((\$x r_isa mammifere) OU (\$x r_isa oiseau)) ET (\$x = ch%))</code>	OU imbriqué
5	<code>(\$x r_isa artiste) ET ((\$x = ba%) OU (\$x = Ba%))</code>	Requête officielle
6	<code>(\$x r_isa animal) ET ((\$x r_has_part aile) OU ...)</code>	ET + OU
7	<code>(\$x r_isa animal) ET (\$y r_isa animal) ET (\$x r_can_eat \$y)</code>	2 variables
8	<code>(chat r_isa \$x)</code>	Direction inverse
9	<code>(chat r_has_part \$x)</code>	Exploration
10	<code>(chat r_has_part \$y) ET (\$y r_isa \$z)</code>	3 variables
11	<code>(\$x r_isa animal) ET (\$x r_has_part \$y) ET (\$y = pa%)</code>	3 vars + filtre
12	<code>(lion r_can_eat \$y) ET (\$y r_isa animal)</code>	Ciblée
13	<code>(\$x r_isa)</code>	Erreur syntaxique
14	<code>(\$x relation_inconnue animal)</code>	Relation invalide

10.3 Analyse des résultats

Les résultats du benchmark sont générés automatiquement dans `benchmark_results.md` après exécution de `node benchmark.js`. Voir ce fichier pour les métriques exactes (durées, appels API, joinStats).

Les observations générales sont :

- Les requêtes simples (1-2 variables) sont traitées en moins de 2s ;
- Le cache réduit significativement les durées des requêtes répétées ;

- La requête officielle à 2 variables (`r_can_eat`) peut retourner 0 résultat si la relation est peu renseignée dans JDM ;
- La pagination est utilisée pour les grandes listes (ex: `r_isa animal` ≥ 500 entrées).

Chapter 11

Limites et perspectives

11.1 Limites actuelles

Densité de JDM. La qualité des résultats dépend directement de la densité du graphe JDM pour les relations utilisées. La relation `r_can_eat` est peu renseignée, ce qui explique que la requête officielle à 2 variables peut retourner peu ou aucun résultat.

Combinatoire des jointures. Pour les requêtes à 2-3 variables, le nombre de candidats à tester peut exploser. Les limites de sécurité (`joinCandidateLimit=500`, `joinEarlyStop=1000`) contrôlent cet espace mais peuvent tronquer les résultats.

Polysémie non gérée. Un mot comme "chat" peut correspondre à plusieurs nœuds JDM (animal, messagerie...). Le moteur prend le nœud canonique sans désambiguïsation.

Pagination bornée. La pagination est limitée à 5000 relations par clause pour protéger l'API du LIRMM. Des relations plus volumineuses sont tronquées.

11.2 Perspectives d'amélioration

- **Désambiguïsation** : exploiter l'endpoint `/refinements/` pour gérer la polysémie ;
- **Index local** : construire un index partiel des relations fréquentes pour réduire les appels API ;
- **Jointures parallèles** : tester plusieurs candidats en parallèle avec `Promise.all` contrôlé ;
- **Requêtes récursives** : explorer des chemins de longueur variable dans le graphe ;
- **Langage naturel** : ajouter une couche de traduction langage naturel $\rightarrow$ requête WIDO.

Chapter 12

Conclusion

Ce projet a conduit à la conception et à l'implémentation complète du moteur sémantique WIDO, capable d'interroger le graphe de connaissances JEUXDEMOTS via un langage de requêtes formel à variables.

Les contributions principales sont :

- Un **parseur** robuste gérant les opérateurs ET, OU, les filtres textuels et les erreurs syntaxiques ;
- Un **planificateur** combinant heuristique structurelle et estimation de cardinalité pour optimiser l'ordre d'exécution ;
- Un **moteur d'exécution** utilisant les IDs JDM pour des jointures exactes et efficaces ;
- Une **pagination API** permettant de récupérer des listes complètes sans surcharger le service ;
- Une **interface glassmorphique** affichant proprement les résultats, les scores, les preuves et les statistiques de diagnostic ;
- Un **benchmark automatique** de 14 requêtes générant des métriques reproductibles.

Le moteur est pleinement fonctionnel et a été validé sur les requêtes officielles du sujet TER. Ses limites principales (densité de JDM, combinatoire des jointures) sont inhérentes à la nature du graphe source et non à l'architecture du moteur.

WIDO constitue une base solide pour des travaux futurs sur la recherche sémantique en langage naturel dans des graphes de connaissances collaboratifs.

Bibliography

- [1] Steve Harris and Andy Seaborne. Sparql 1.1 query language. *W3C Recommendation*, 2013.
- [2] Natalya F. Noy and Deborah L. McGuinness. *Ontology Development 101: A Guide to Creating Your First Ontology*. Stanford Knowledge Systems Laboratory, 2009.
- [3] Patricia G. Selinger et al. Access path selection in a relational database management system. In *Proceedings of SIGMOD*, pages 23–34, 1979.